

## Algoritmi e Strutture Dati – 28/08/2026

### Esercizio A1 – Complessità

**Idea chiave** Il costo del ciclo interno dipende dall'ordine degli elementi. Per ottenere un caso asintotico non si sceglie un singolo valore di  $n$ : si descrive invece una famiglia di vettori, uno per ogni dimensione  $n$ .

- Se  $A$  è ordinato in senso non decrescente, per ogni  $i$  il test  $A[i+1] < A[i]$  è falso. Il ciclo interno non esegue il proprio corpo, ma il ciclo esterno esegue comunque  $n - 1$  iterazioni: il costo è  $\Theta(n)$ .
- Se  $A$  è ordinato in senso strettamente decrescente, per un dato  $i$  il ciclo interno esegue  $n - i - 1$  iterazioni. Il lavoro complessivo è

$$\sum_{i=0}^{n-2} (n - i - 1) = (n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2} = \Theta(n^2).$$

### Se ti riconosci in uno di questi ragionamenti

**“Nel caso ottimo scelgo  $n = 2$ , quindi il costo è costante.”** Un caso ottimo asintotico deve valere per dimensioni arbitrariamente grandi. Scegliere un solo valore di  $n$  elimina artificialmente la crescita che si vuole misurare. Per migliorare la risposta, descrivi una famiglia di input (qui: i vettori non decrescenti) e conta il lavoro per un generico  $n$ .

**“La somma dei cicli interni è quadratica; poi la moltiplico per le iterazioni del ciclo esterno.”** La sommatoria contiene già il contributo di *tutte* le iterazioni del ciclo esterno. Moltiplicare ancora per  $n$  conta due volte lo stesso ciclo. Per evitarlo, scrivi prima il costo interno relativo a un indice  $i$  e poi somma su tutti i valori di  $i$  una sola volta.

**“ $(n - 1) + (n - 2) + \dots + 1$  è un fattoriale.”** Il fattoriale nasce da un prodotto,  $n(n - 1) \dots 1$ ; qui abbiamo una somma. Riconduci la sequenza alla somma dei primi  $n - 1$  interi oppure usa la formula  $n(n - 1)/2$ .

**“Scrivo soltanto  $\Theta(n)$  e  $\Theta(n^2)$ .”** Il risultato è corretto soltanto se è collegato agli input che lo realizzano. Una motivazione breve ma completa contiene sempre: forma dell'input, numero di iterazioni del ciclo interno e somma finale.

**Controllo finale** Chiediti se il caso scelto è definito per ogni  $n$  e se ogni iterazione è stata contata esattamente una volta.

## Esercizio A2 – Cavallo su scacchiera

**Idea chiave** Consideriamo ogni casella come un nodo di un grafo non orientato. Due nodi sono adiacenti quando il cavallo può spostarsi dall'una all'altra casella con una sola mossa. Tutti gli archi hanno lo stesso costo, quindi una visita in ampiezza a partire dalla sorgente calcola il numero minimo di mosse necessario per raggiungere ogni casella visitata.

---

```
int minKnightMoves(int n, int startRow, int startCol, int endRow, int endCol)
{
    precondition: n > 0 and 0 ≤ startRow, startCol, endRow, endCol < n
    int[] rowMove = [-2, -2, -1, -1, 1, 1, 2, 2]
    int[] colMove = [-1, 1, -2, 2, -2, 2, -1, 1]
    int[][] distance = new int[n][n] { -1 }
    QUEUE Q = Queue()
    distance[startRow][startCol] = 0
    Q.enqueue((startRow, startCol))
    while not Q.isEmpty() do
        int r, c = Q.dequeue()
        if r == endRow and c == endCol then
            return distance[r][c]
        for k = 0 to 7 do
            int nextRow = r + rowMove[k]
            int nextCol = c + colMove[k]
            if 0 ≤ nextRow < n and 0 ≤ nextCol < n and distance[nextRow][nextCol] == -1 then
                distance[nextRow][nextCol] = distance[r][c] + 1
                Q.enqueue((nextRow, nextCol))
    return -1
}
```

---

Quando una casella viene inserita nella coda, il valore memorizzato in *distance* è la lunghezza di un cammino minimo dalla sorgente a tale casella: la BFS esamina infatti le caselle in ordine non decrescente di distanza. Per questo motivo, quando viene estratta la destinazione, il valore restituito è il numero minimo di mosse. Se la coda si svuota, tutte le caselle raggiungibili sono state visitate e la destinazione non può essere raggiunta.

In realtà, per  $n \geq 4$  il cavallo può raggiungere qualunque casella della scacchiera a partire da qualunque altra casella, quindi in questi casi l'algoritmo non restituisce mai  $-1$ . I casi più piccoli sono particolari: per  $n = 1$  esiste una sola casella; per  $n = 2$  non è possibile effettuare alcuna mossa; per  $n = 3$  la casella centrale è isolata, mentre le altre otto appartengono alla stessa componente connessa. Il valore  $-1$  serve dunque a gestire correttamente questi ultimi due casi.

La scacchiera contiene  $n^2$  caselle e da ciascuna di esse si esaminano otto possibili mosse. La complessità nel caso pessimo è quindi  $\Theta(n^2)$ .

### Se ti riconosci in uno di questi tentativi

**“Uso una coda e incremento un contatore ogni volta che estraggo una casella.”** La coda contiene spesso molte caselle alla stessa distanza. Il numero di estrazioni misura quanti nodi sono stati esaminati, non quante mosse separano la sorgente dalla casella corrente. Associa invece una distanza a ogni casella, oppure inserisci in coda la coppia (casella, distanza), e assegna a un vicino non visitato la distanza corrente più uno.

**“Ho scritto che serve una BFS, ma non ho specificato l'algoritmo.”** L'idea è quella giusta, ma non permette ancora di verificare inizializzazione, mosse, gestione dei visitati e valore restituito. Trasforma il nome della tecnica in passi eseguibili: stato della coda, distanza iniziale, generazione dei vicini, marcatura e condizione di arresto.

**“Modifico direttamente  $r$  e  $c$  mentre genero gli otto vicini.”** Dopo la prima mossa, le mosse successive partirebbero dalla casella appena modificata e non più dalla casella estratta. Conserva  $r, c$  e calcola ogni volta nuove variabili  $nextRow, nextCol$ .

**“Marco una casella come visitata quando la estraggo.”** La stessa casella può essere inserita più volte da predecessori differenti, aumentando lavoro e memoria e rendendo più difficile ragionare sulla distanza. Nella BFS conviene marcarla quando viene *inserita*: la prima scoperta avviene già lungo un cammino minimo.

**“Provo ricorsivamente tutti i cammini e tengo il più corto.”** Una ricerca esaustiva può riesaminare un numero enorme di cammini e, senza una gestione impeccabile dei visitati, può anche entrare in ciclo. Qui non serve enumerare i cammini: poiché ogni mossa costa uno, la BFS li considera per lunghezza crescente e si ferma appena raggiunge la destinazione. La visita richiede  $\Theta(n^2)$  nel caso pessimo; enumerare cammini semplici può invece richiedere tempo esponenziale.

**“Uso le mosse del cavallo, ma il risultato non torna.”** Controlla che gli spostamenti siano esattamente le otto coppie con valori assoluti  $(1, 2)$  oppure  $(2, 1)$ , che riga e colonna usino indici abbinati e che i limiti siano verificati *prima* di accedere alla matrice.

**Controllo finale** Per ogni casella estratta, verifica tre invarianti: la sua distanza è minima; ogni vicino viene calcolato a partire dalla casella estratta; ogni casella entra in coda al più una volta.

## Esercizio A3 – Sottoalbero ABR massimo

**Idea chiave** La funzione ricorsiva restituisce una tupla  $(min, max, size, isBST)$ . Se il sottoalbero corrente è un ABR,  $min$  e  $max$  sono la chiave minima e massima in esso contenute e  $size$  è il suo numero di nodi. Se non è un ABR,  $size$  è invece la dimensione del più grande ABR trovato al suo interno; in questo caso  $min$  e  $max$  non vengono utilizzati.

---

```
int largestBST(TREE T)
```

---

```
    int, int, int, boolean min, max, size, isBST = largestBSTRec(T)
    return size
```

---

---

```
(int, int, int, boolean) largestBSTRec(TREE t)
```

---

```
    if t == nil then
        return  $\langle +\infty, -\infty, 0, \text{true} \rangle$ 

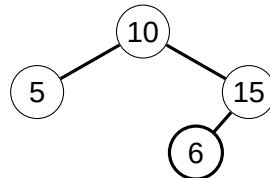
    int, int, int, boolean minL, maxL, sizeL, isBSTL = largestBSTRec(t.left)
    int, int, int, boolean minR, maxR, sizeR, isBSTR = largestBSTRec(t.right)
    if isBSTL and isBSTR and maxL < t.key and t.key < minR then
        return  $\langle \min(min_L, t.key), \max(t.key, max_R), size_L + size_R + 1, \text{true} \rangle$ 
    else
        return  $\langle 0, 0, \max(size_L, size_R), \text{false} \rangle$ 
```

---

Il sottoalbero vuoto viene considerato un ABR di dimensione 0; i valori  $+\infty$  e  $-\infty$  fanno funzionare senza casi speciali il controllo di un nodo con uno o due figli vuoti. Un nodo  $t$  è radice di un ABR esattamente quando entrambi i suoi sottoalberi sono ABR e la massima chiave a sinistra è minore di  $t.key$ , che a sua volta è minore della minima chiave a destra. In tal caso le dimensioni si sommano; altrimenti il più grande ABR è quello già trovato nel sottoalbero sinistro oppure in quello destro.

La funzione visita ogni nodo una sola volta ed esegue un numero costante di operazioni per nodo; la complessità è quindi  $\Theta(n)$ .

**Perché controllare soltanto padre e figli non basta** Considera l'albero



Ogni confronto immediato sembra corretto:  $5 < 10$ ,  $15 > 10$  e  $6 < 15$ . Tuttavia 6 appartiene al sottoalbero destro di 10 e dovrebbe quindi essere maggiore di 10. Per certificare la radice occorre conoscere gli estremi dell'intero sottoalbero sinistro e destro, non soltanto le chiavi dei figli.

**Se ti riconosci in uno di questi tentativi**

**“Confronto la chiave del nodo con quelle dei suoi figli.”** Questo verifica soltanto relazioni locali e non vede una violazione prodotta da un discendente più lontano. Fai restituire a ogni chiamata il minimo e il massimo di tutto il sottoalbero; confronta  $max_L < t.key < min_R$ .

**“Restituisco dimensione e una chiave del figlio.”** La chiave della radice di un figlio non è necessariamente l'estremo del suo sottoalbero. Il padre ha bisogno del massimo a sinistra e del minimo a destra; per poter propagare a sua volta l'intervallo servono entrambi gli estremi.

**“Propago minimo, massimo e dimensione, ma non un booleano.”** Gli estremi non dicono se il sottoalbero era già invalido. Senza  $isBST$  un antenato può combinare nuovamente un sottoalbero non ABR come se fosse valido. Aggiungi un'informazione esplicita di validità e consenti la combinazione soltanto quando entrambi i figli sono ABR.

**“Quando un nodo non è valido restituisco subito zero o  $-1$ .”** La radice corrente può non formare un ABR, ma uno dei suoi discendenti può essere la radice della soluzione migliore. Nel ramo non valido restituisci quindi il massimo fra le migliori dimensioni già trovate a sinistra e a destra.

**“Uso un contatore globale che cresce o si azzerà lungo la visita.”** Un sottoalbero comprende simultaneamente entrambi i rami, mentre un contatore di cammino descrive soltanto la sequenza di nodi attraversata. Ogni chiamata deve produrre un riassunto autonomo del proprio sottoalbero; il padre combina poi i due riassunti.

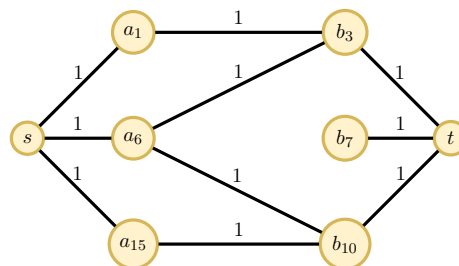
**“Per ogni nodo verifico da capo se tutto il suo sottoalbero è un ABR.”** La risposta può essere corretta, ma gli stessi nodi vengono riletti molte volte e il costo può diventare  $\Theta(n^2)$ . La tupla restituita dalla visita post-order evita il lavoro ripetuto e porta la complessità a  $\Theta(n)$ .

**Controllo finale** Prima di scrivere il codice, formula l'invariante della funzione ricorsiva: “la tupla descrive completamente il sottoalbero radicato in  $t$ ”. Verifica poi separatamente il caso vuoto, il caso ABR e il caso non ABR.

## Esercizio B1 – Numeri inquadrabili

Il problema corrisponde alla ricerca di un matching massimo in un grafo bipartito e può essere risolto con la seguente rete di flusso:

- si creano una sorgente  $s$  e un pozzo  $t$ ;
- per ogni  $x \in X$  si crea un nodo  $a_x$  e l'arco  $(s, a_x)$  di capacità 1;
- per ogni  $y \in Y$  si crea un nodo  $b_y$  e l'arco  $(b_y, t)$  di capacità 1;
- si crea l'arco  $(a_x, b_y)$  di capacità 1 se e solo se  $x$  e  $y$  sono inquadrabili.



Ogni insieme di  $k$  coppie inquadrabili che non riutilizza alcun valore definisce un flusso di valore  $k$ : per ogni coppia  $(x, y)$  si invia un'unità lungo il cammino  $s, a_x, b_y, t$ . Viceversa, poiché le capacità sono intere, esiste un flusso massimo intero. Gli archi  $(a_x, b_y)$  con flusso 1 individuano coppie inquadrabili; le capacità degli archi adiacenti a  $s$  e  $t$  assicurano che ciascun valore venga usato al più una volta. Il valore del flusso massimo coincide quindi con il massimo numero di coppie richiesto.

Posto  $|X| + |Y| = 2n$ , la rete contiene  $2n + 2$  nodi e al più  $|X||Y| + 2n = \mathcal{O}(n^2)$  archi. Infatti, da  $|X| + |Y| = 2n$  segue  $|X||Y| \leq n^2$ . Il flusso massimo vale al più  $\min(|X|, |Y|) \leq n$ ; usando Ford–Fulkerson, ogni cammino aumentante viene trovato in  $\mathcal{O}(|V| + |E|) = \mathcal{O}(n^2)$  tempo e la complessità è quindi  $\mathcal{O}(n^3)$ . La costruzione della rete richiede inoltre di controllare tutte le coppie in  $X \times Y$ , con costo  $\mathcal{O}(n^2)$ .

**Perché una soluzione greedy non è corretta** Scegliere una qualunque coppia inquadrabile e rimuoverne subito i due valori non è corretto. Un controesempio minimo è

$$X = [6, 1], \quad Y = [3, 10].$$

Le sole coppie inquadrabili sono

$$(6, 3), \quad (6, 10), \quad (1, 3),$$

perché  $6+3 = 9$ ,  $6+10 = 16$  e  $1+3 = 4$  sono quadrati, mentre  $1+10 = 11$  non lo è. Se il greedy sceglie per prima  $(6, 3)$ , consuma contemporaneamente l'unico elemento di  $X$  che può essere abbinato a 10 e l'unico elemento di  $Y$  che può essere abbinato a 1: non resta quindi alcuna seconda coppia. La soluzione ottima sceglie invece le due coppie disgiunte  $(6, 10)$  e  $(1, 3)$ . Una scelta localmente valida può dunque impedire il matching globale massimo.

**Perché una ricorrenza in stile LCS non basta** Una soluzione basata sulla LCS è più sistematica di un greedy, ma introduce un vincolo che il problema non possiede: conserva l'ordine dei due vettori. Nell'esempio precedente, le coppie della soluzione ottima sono  $(X[0], Y[1]) = (6, 10)$  e  $(X[1], Y[0]) = (1, 3)$ , quindi i due archi si incrociano rispetto agli indici. Una ricorrenza LCS, dopo aver scelto una coppia, consuma i due elementi e continua soltanto su prefissi o suffissi compatibili con quella scelta; non può quindi rappresentare entrambe le coppie incrociate. Per questo, rispetto al problema dato, non supera davvero il limite della scelta greedy: rende definitiva una compatibilità locale e scarta abbinamenti globali che potrebbero essere migliori.

**Perché provare tutte le possibilità è eccessivo** Un algoritmo esaustivo può naturalmente trovare il matching massimo, ma non è una soluzione efficiente. Provare tutti i sottoinsiemi dei possibili archi richiede fino a  $2^{|X||Y|}$  tentativi; anche un backtracking più accorto, che associa a ogni elemento di  $X$  un elemento non ancora usato di  $Y$ , può esplorare un numero fattoriale di abbinamenti. Il tempo è quindi almeno esponenziale nel caso peggiore, mentre la riduzione a flusso massimo fornisce qui un algoritmo polinomiale in  $\mathcal{O}(n^3)$ .

## Esercizio B2 – Dismutazioni

La soluzione segue lo stesso schema usato per generare tutte le permutazioni. Il vettore booleano *used* indica quali valori sono già presenti nel prefisso  $P[0 \dots curr - 1]$ . Quando si riempie la posizione *curr*, si può scegliere qualunque valore non ancora usato e diverso da *curr*.

---

```
printDismutations(int n)
  precondition:  $n \geq 0$ 
  int[] P = new int[0...n - 1]
  boolean[] used = new boolean[0...n - 1]
  for v = 0 to n - 1 do
    | used[v] = false
  printDismutationRec(P, used, n, 0)
```

---

```
printDismutationRec(int[] P, boolean[] used, int n, int curr)
  precondition:  $0 \leq curr \leq n$ 
  if curr == n then
    | print P
  else
    for v = 0 to n - 1 do
      | if not used[v] and  $v \neq curr$  then
        |   P[curr] = v
        |   used[v] = true
        |   printDismutationRec(P, used, n, curr + 1)
        |   used[v] = false
```

---

Ad ogni chiamata, il prefisso già costruito contiene valori distinti e nessuno di essi coincide con la propria posizione. Marcare il valore scelto impedisce di usarlo due volte; smarcarlo al ritorno permette di provare tutte le altre scelte. Quando  $curr = n$ ,  $P$  è quindi una dismutazione. Viceversa, ogni dismutazione determina una sequenza di scelte esplorata dall'algoritmo, perciò vengono stampate tutte e sole le soluzioni.

Per la valutazione del compito, è ragionevole osservare che ogni dismutazione è una delle  $n!$  permutazioni. Il numero di soluzioni è dunque  $O(n!)$ : è un limite superiore corretto, anche se non fornisce un limite inferiore. Poiché stampare ogni vettore costa  $O(n)$ , ne segue  $O(n \cdot n!)$  per l'algoritmo. Come approfondimento, il numero  $D_n$  di dismutazioni (in inglese, *derangements*) è

$$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}.$$

Poiché  $D_n \sim n!/e$ , ovvero  $D_n/(n!/e)$  tende a 1, si ha  $D_n = \Theta(n!)$ . L'output contiene pertanto  $\Theta(n \cdot n!)$  valori e richiede altrettanto tempo per essere stampato. Per ulteriori dettagli si veda la voce Wikipedia [Dismutazione \(matematica\)](#).

## Esercizio B3 – Pari-bilanciato

**Soluzione cubica** La soluzione più semplice enumera tutti i sottovettori  $A[start \dots end]$  e, per ciascuno di essi, conta da capo gli elementi pari e dispari.

---

```
int maxBalancedCubic(int[] A, int n)
```

---

```
precondition:  $n \geq 0$ 
int best = 0
for start = 0 to  $n - 1$  do
    for end = start to  $n - 1$  do
        int even = 0
        int odd = 0
        for i = start to end do
            if  $A[i] \bmod 2 == 0$  then
                even = even + 1
            else
                odd = odd + 1
        if even == odd then
            best = max(best, end - start + 1)
    return best
```

---

Ogni sottovettore viene esaminato e la sua parità viene calcolata correttamente, quindi il massimo memorizzato è la risposta richiesta. I tre cicli annidati hanno complessità  $\mathcal{O}(n^3)$ .

**Soluzione quadratica con somme prefisse** Associamo  $+1$  a ogni elemento pari e  $-1$  a ogni elemento dispari. Definiamo  $prefix[i]$  come la somma dei valori associati ai primi  $i$  elementi di  $A$ , con  $prefix[0] = 0$ . Il sottovettore  $A[i \dots j - 1]$  è pari-bilanciato se e solo se  $prefix[i] = prefix[j]$ .

---

```
int maxBalancedQuadratic(int[] A, int n)
```

---

```
precondition:  $n \geq 0$ 
int[] prefix = new int[0 .. n]
prefix[0] = 0
for i = 0 to  $n - 1$  do
    if  $A[i] \bmod 2 == 0$  then
        prefix[i + 1] = prefix[i] + 1
    else
        prefix[i + 1] = prefix[i] - 1
int best = 0
for i = 0 to  $n - 1$  do
    for j = i + 1 to n do
        if prefix[i] == prefix[j] then
            best = max(best, j - i)
return best
```

---

L'uguaglianza fra due somme prefisse equivale ad avere somma zero fra i due indici, cioè lo stesso numero di elementi pari e dispari. L'algoritmo prova tutte le coppie di estremi e ha complessità  $\mathcal{O}(n^2)$ .

**Soluzione con tabella hash** La ricerca di due somme prefisse uguali può essere resa più efficiente. La tabella hash *first* associa a ogni bilanciamento la lunghezza del primo prefisso in cui esso è comparso. Conservare la prima occorrenza permette di ottenere il sottovettore più lungo che termina nella posizione corrente.

---

**int** maxBalanced(**int**[] *A*, **int** *n*)

---

**precondition:**  $n \geq 0$ HASH *first* = Hash()*first*.insert(0, 0)**int** *balance* = 0**int** *best* = 0**for** *i* = 0 **to** *n* - 1 **do**    **if** *A*[*i*] mod 2 == 0 **then**        | *balance* = *balance* + 1    **else**        | *balance* = *balance* - 1    **int** *prefixLength* = *i* + 1    **if** *first*.lookup(*balance*) == nil **then**        | *first*.insert(*balance*, *prefixLength*)    **else**        | *best* = max(*best*, *prefixLength* - *first*.lookup(*balance*))    **return** *best*

---

La coppia iniziale (0, 0) rappresenta il prefisso vuoto. Quando il bilanciamento corrente era già comparso, il sottovettore compreso fra la prima occorrenza e la posizione corrente è pari-bilanciato ed è il più lungo con quell'estremo destro. Assumendo costo atteso costante per le operazioni sulla tabella hash, la complessità attesa è  $\mathcal{O}(n)$ .